

다항식 및 희소 행렬 처리 방법 정리

1. 다항식 표현 및 연산

1.1 다항식 표현 방법

다항식을 프로그램에서 표현하기 위해서는 자료구조가 필요하다. 이를 배열을 이용하여 표현하는 방식은 크게 두 가지가 있다.

(1) 모든 항을 저장하는 방법

- 다항식의 차수별 계수를 배열에 저장한다.
- 예시:
 - 다항식: $(7x^5 + 0x^4 + 0x^3 + 3x^0)$
 - 배열 표현: `[7, 0, 0, 0, 0, 3]`
- **장점:** 연산이 단순해짐.
- **단점:** 계수가 0인 항까지 저장해야 하므로 메모리 낭비 발생.

(2) 0이 아닌 항만 저장하는 방법

- 계수와 차수를 각각 저장하는 방식.
- 예시:
 - 다항식: $(7x^5 + 3x^0)$
 - 배열 표현:

계수	차수
7	5
3	0

- **장점:** 메모리 효율이 높음.
- **단점:** 연산이 다소 복잡해짐.

1.2 다항식 덧셈 연산

두 다항식을 더할 때, 같은 차수의 항을 더하고, 나머지는 그대로 유지한다.

예시

- 다항식 1: $(3x^5 + 6x^4 + 2x^2)$
- 다항식 2: $(7x^4 + 5x^2 + x^0)$
- 덧셈 결과: $(3x^5 + (6+7)x^4 + (2+5)x^2 + x^0 = 3x^5 + 13x^4 + 7x^2 + x^0)$

2. 희소 행렬 표현 및 연산

2.1 희소 행렬(Sparse Matrix)

희소 행렬이란, 대부분의 원소가 0인 행렬을 의미하며, 이를 저장할 때 0을 제외한 값만 저장하는 방식이 필요하다.

2.2 희소 행렬의 표현 방법

(1) 2차원 배열을 이용하는 방법

- 전체 행렬을 저장하므로 연산이 간단함.
- 하지만 대부분이 0인 경우, 메모리 낭비가 심함.

(2) 0이 아닌 요소만 저장하는 방법

- 행, 열, 값을 저장하는 방식.
- 예시 (행렬):

```
| 2 | 3 | 0 | 8 |  
| 9 | 1 | 7 | 0 |  
| 0 | 5 | 0 | 6 |
```

- 배열 표현:

```
| 행 | 열 | 값 |  
|----|----|----|  
| 0 | 0 | 2 |  
| 0 | 1 | 3 |  
| 0 | 3 | 8 |  
| 1 | 0 | 9 |  
| 1 | 1 | 1 |  
| 1 | 2 | 7 |  
| 2 | 1 | 5 |  
| 2 | 3 | 6 |
```

- **장점:** 메모리 사용량 절약.
- **단점:** 연산이 복잡해짐.

2.3 희소 행렬 전치(Transpose) 연산

희소 행렬을 전치(transpose)할 때, 행과 열을 서로 바꾸어 저장한다.

예시

- 원본 행렬:

```
| 2 3 0 8 |
| 9 1 7 0 |
| 0 5 0 6 |
```

- 전치 행렬:

```
| 2 9 0 |
| 3 1 5 |
| 0 7 0 |
| 8 0 6 |
```

- 배열 표현 (행과 열을 바꾼 후 저장):

행	열	값
0	0	2
1	0	3
3	0	8
0	1	9
1	1	1
2	1	5
1	2	7
3	2	6

3. 포인터(Pointer) 개념

3.1 포인터 기본 개념

- **포인터란?** 변수가 저장된 메모리 주소를 저장하는 변수.
- 예시:

```
int a = 10;
int *p = &a; // p는 a의 주소를 저장
```

3.2 포인터 연산

- 포인터는 특정 변수의 주소를 가리키며, 해당 주소의 값을 참조할 수 있다.
- *p를 이용하여 값을 변경 가능.

```
*p = 20; // a의 값이 20으로 변경됨
```

3.3 함수에서 포인터 사용

포인터를 함수의 매개변수로 사용하면, 함수 내부에서 변수의 값을 변경할 수 있음.

예제: 두 변수 값 교환

```
void swap(int *x, int *y) {  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```

- `swap(&a, &b);` 를 호출하면 `a`와 `b`의 값이 바뀜.

4. 결론

1. **다항식 연산**에서는 메모리 절약을 위해 0이 아닌 항만 저장하는 방법이 효율적이다.
2. **희소 행렬 연산**에서도 0이 아닌 요소만 저장하는 방법이 메모리 절약에 유리하지만, 연산이 복잡해진다.
3. **포인터 개념**을 활용하면 변수의 주소를 직접 조작하여 메모리를 효율적으로 사용할 수 있다.

이론과 실습을 병행하여 연산의 개념과 구현을 익혀야 한다.